



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие №8

Студенты группы *ИКБО-20-23 Комисарик М.А.*

(подпись)

Ассистент *Брайловский А.В.*

(подпись)

Отчет представлен «__»_____2025 г.

Москва 2025 г.

ПОСТАНОВКА ЗАДАЧИ

Цель работы: Освоение методов работы со сложными и неструктурированными типами данных (JSONB, BYTEA) в PostgreSQL, а также получение практических навыков реализации горизонтального масштабирования базы данных с помощью декларативного секционирования.

Постановка задачи:

1. Хранение паролей (BYTEA).

- 1) Создать справочную таблицу ролей и таблицу системных пользователей.
- 2) В таблице пользователей предусмотреть поля для хранения «сырого» пароля и его хеша (тип BYTEA).
- 3) Сохранить хеши паролей пользователей, используя функцию хеширования digest.
- 4) Выполнить запрос, проверяющий соответствие введенного пароля сохраненному хешу. Одна проверка должна быть успешной, другая - нет.

2. Секционирование таблицы логов.

- 1) Создать новую секционированную таблицу для хранения логов событий.
- 2) Использовать стратегию секционирования по диапазонам (RANGE) по полю даты.
- 3) Создать две секции для периодов:
 - 2-е полугодие 2024 (с 2024-07-01 по 2025-01-01).
 - 1-е полугодие 2025 (с 2025-01-01 по 2025-07-01).
- 4) Создать секцию по умолчанию, куда будут попадать все данные, выходящие за рамки указанных диапазонов.

3. Генерация данных.

Написать скрипт на PL/pgSQL для генерации 20 000 записей в диапазоне дат, покрывающем созданные секции. Сгенерированные данные должны содержать JSON-структуру с различным набором полей для разных типов

событий (например, чтобы логи «продажи» отличались по структуре от логов «входа»). Вывести сгенерированные данные – показать запросом SELECT, что данные успешно созданы и физически распределились по разным таблицам секциям.

4. Анализ и поиск по JSONB.

Написать три запроса:

1) Фильтрация по значению – найти записи, где числовое поле внутри JSON удовлетворяет математическому условию (например, > 800), используя операторы извлечения -> и ->>.

2) Поиск по вхождению – найти записи, содержащие точный фрагмент JSON-структуры (например, {"quantity": 5}), используя оператор @>.

3) Агрегация – посчитать сумму или среднее значение числового поля из JSON-документа.

5. Модификация данных JSONB.

Продemonстрировать изменение данных внутри JSON-поля:

1) Массовое добавление нового поля во все записи определенного типа.

2) Точечное изменение значения по ключу в конкретной записи.

ХОД РАБОТЫ

Начальные данные

	123 id_employee	123 id_job_position	A-Z first_name	A-Z second_name	A-Z third_name	A-Z app_account_password_hash	A-Z phone_number	
1	1	1	Alice	Johnson	[NULL]	emp_hash_1	555-3333	1
2	2	2	Bob	Williams	Lee	emp_hash_2	555-4444	2
3	3	3	Carol	Martinez	[NULL]	emp_hash_3	555-8888	3
4	4	4	Dave	Anderson	Paul	emp_hash_4	555-9999	4
5	5	5	Eva	Garcia	Maria	emp_hash_5	555-0000	5
6	6	2	Frank	Taylor	[NULL]	emp_hash_6	555-1212	6
7	7	3	Grace	Thomas	Elizabeth	emp_hash_7	555-1313	7

Рисунок 1 – Содержание таблицы employee

	123 id_job_position	A-Z name	A-Z salary	
1	1	Manager	50000	
2	2	Chef	45000	
3	3	Delivery Driver	30000	
4	4	Cashier	28000	
5	5	Assistant Manager	42000	

Рисунок 2 – Содержание таблицы job_position

	123 id_product	A-Z name	A-Z description	123 price	
1	1	Margherita Pizza	Classic pizza with tomato and cheese	\$10.99	
2	2	Pepperoni Pizza	Pizza with pepperoni slices	\$12.99	
3	3	Cola	Refreshing soft drink	\$2.99	
4	4	Veggie Supreme	Pizza with assorted vegetables	\$14.99	
5	5	Chocolate Cake	Rich chocolate dessert	\$6.99	
6	6	Caesar Salad	Fresh salad with Caesar dressing	\$8.99	
7	7	Garlic Bread	Toasted bread with garlic butter	\$4.99	
8	8	Sprite	Sprite	\$3.49	
9	9	Lipton tea	A 1 liter bottle of Lipton tea	\$4.49	
10	10	Chocolate muffin	Tasty little chocolate dessert	\$4.99	
11	11	Chocolate donut	A chocolate glazed donut	\$5.49	

Рисунок 3 – Содержание таблицы product

	123 id_category	A-Z name	A-Z description	
1	1	Pizza	Various types of pizzas	
2	2	Beverage	Drinks and beverages	
3	3	Appetizer	Starters and snacks	
4	4	Dessert	Sweet treats	
5	5	Salad	Fresh salads	

Рисунок 4 – Содержание таблицы category

	123 id_product_ingredient	123 id_product	123 id_ingredient	123 ingredient_weight
1	1	1	1	500
2	2	1	2	200
3	3	1	3	150
4	4	2	1	500
5	5	2	2	200
6	6	2	3	150
7	7	2	4	100
8	8	4	1	500
9	9	4	2	200
10	10	4	3	150
11	11	4	5	80
12	12	4	6	70
13	13	4	7	60

Рисунок 5 – Содержание таблицы product_ingredient

	123 id_product_category	123 id_product	123 id_category
1	1	1	1
2	2	2	1
3	3	3	2
4	4	4	1
5	5	5	4
6	6	6	5
7	7	7	3
8	8	8	2
9	9	9	2
10	10	10	4
11	11	11	4

Рисунок 6 – Содержание таблицы product_category

	123 id_ingredient	A-Z name
1	1	Flour
2	2	Cheese
3	3	Tomato Sauce
4	4	Pepperoni
5	5	Mushrooms
6	6	Onions
7	7	Olives

Рисунок 7 – Содержание таблицы ingredient

	123 id_pizzeria	A-Z phone_number	123 id_address
1	1	555-0001	1
2	2	555-0002	2
3	3	555-0003	3
4	4	555-0004	4
5	5	555-0005	5

Рисунок 8 – Содержание таблицы pizzeria

	123 id_pizzeria_storage	123 id_pizzeria	123 id_storage	
1	1	1	1	
2	2	2	2	
3	3	3	3	
4	4	4	4	
5	5	5	5	

Рисунок 9 – Содержание таблицы pizzeria_storage

	123 id_storage_ingredient	123 id_storage	123 id_ingredient	123 ingredient_weight
1	4	2	4	2 000
2	5	3	5	1 500
3	6	4	6	1 800
4	7	5	7	1 200
5	2	1	2	5 000
6	1	1	1	10 000
7	3	1	3	3 000

Рисунок 10 – Содержание таблицы storage_ingredient

	123 id_client	AZ first_name	AZ second_name	AZ third_name	AZ phone_number	123 id_delivery_address	AZ account_password_id_hash
1	1	John	Doe	[NULL]	555-1111	1	hashed_password_123
2	2	Jane	Smith	Marie	555-2222	2	hashed_password_456
3	3	Mike	Johnson	Robert	555-3333	3	hashed_password_789
4	4	Sarah	Wilson	[NULL]	555-4444	4	hashed_password_101
5	5	David	Brown	James	555-5555	5	hashed_password_112
6	6	Emily	Davis	Anne	555-6666	6	hashed_password_131
7	7	Chris	Miller	Thomas	555-7777	7	hashed_password_415

Рисунок 11 – Содержание таблицы client

	123 id_address	AZ city	AZ street	123 house_number	123 floor	123 unit_number
1	1	New York	Main St	123	[NULL]	[NULL]
2	2	Los Angeles	Oak Ave	456	3	5
3	3	Chicago	Pine Rd	789	2	[NULL]
4	4	Boston	Maple St	321	1	2
5	5	Seattle	Cedar Ave	654	4	8
6	6	Miami	Palm Blvd	987	[NULL]	[NULL]
7	7	Denver	Elm St	147	2	3

Рисунок 12 – Содержание таблицы address

	123 id_order	123 id_pizzeria	123 id_client	123 id_delivery_address	123 order_amount	date_of_formation	123 delivery_method_id	123
1	4	1	4	4	\$18.98	2025-09-08	1	1
2	8	1	1	1	\$38.97	2025-11-21	2	1
3	11	1	4	4	\$45.97	2025-06-16	1	2
4	12	1	5	5	\$32.97	2025-06-10	2	1
5	1	1	1	1	\$25.97	2025-11-05	2	3
6	2	1	2	2	\$15.98	2025-02-06	1	2
7	3	1	3	3	\$32.97	2025-09-07	2	3
8	5	1	5	5	\$45.96	2025-02-09	2	2
9	6	1	6	6	\$12.99	2025-06-10	1	3
10	7	1	7	7	\$28.97	2025-09-11	1	4
11	9	1	2	2	\$22.98	2025-02-07	1	2
12	10	1	3	3	\$19.98	2025-11-21	2	1
13	15	1	1	1	\$32.97	2025-11-03	1	3

Рисунок 13 – Содержание таблицы _order.

	123 id_order_product	123 id_order	123 id_product	123 product_count
1	1	1	1	2
2	2	1	3	1
3	3	2	2	1
4	4	3	4	2
5	5	3	3	2
6	6	4	6	1
7	7	4	7	2
8	8	5	5	3
9	9	5	3	3
10	10	6	2	1
11	11	7	1	1
12	12	7	4	1
13	13	7	3	2
14	14	8	2	2
15	15	8	3	2
16	16	9	4	1
17	17	9	7	3
18	18	10	6	2
19	19	10	8	1
20	20	11	1	3
21	21	11	3	1
22	22	12	5	2
23	23	12	9	2
24	24	12	11	1

Рисунок 14 – Содержание таблицы order_product.

1 Хранение паролей

Добавим столбец password_raw в таблицу employee (Рисунок 15).

```

ALTER TABLE employee
ADD COLUMN password_raw TEXT;

```

Статистика 1

Вывод

Name	Value
Updated Rows	0
Execute time	0,041s
Start time	Mon Dec 08 09:24:38 MSK 2025
Finish time	Mon Dec 08 09:24:38 MSK 2025
Query	ALTER TABLE employee ADD COLUMN password_raw TEXT

Рисунок 15 – Создание столбца с нехэшированным паролем

<pre> ALTER TABLE employee ALTER COLUMN app_account_password_hash DROP NOT NULL; UPDATE employee SET app_account_password_hash = NULL; ALTER TABLE employee ALTER COLUMN app_account_password_hash TYPE BYTEA USING app_account_password_hash::bytea; </pre>	Статистика 1 <table> <tr> <th>Name</th><th>Value</th></tr> <tr> <td>Queries</td><td>3</td></tr> <tr> <td>Updated Rows</td><td>7</td></tr> <tr> <td>Execute time</td><td>0,16s</td></tr> <tr> <td>Fetch time</td><td>0s</td></tr> <tr> <td>Total time</td><td>0,16s</td></tr> <tr> <td>Start time</td><td>2025-12-08 09:31:07.581</td></tr> <tr> <td>Finish time</td><td>2025-12-08 09:31:07.744</td></tr> </table>	Name	Value	Queries	3	Updated Rows	7	Execute time	0,16s	Fetch time	0s	Total time	0,16s	Start time	2025-12-08 09:31:07.581	Finish time	2025-12-08 09:31:07.744
Name	Value																
Queries	3																
Updated Rows	7																
Execute time	0,16s																
Fetch time	0s																
Total time	0,16s																
Start time	2025-12-08 09:31:07.581																
Finish time	2025-12-08 09:31:07.744																

Рисунок 16 – Изменение типа столбца с хэшами паролей

<pre> UPDATE employee SET password_raw = 'PizzaWorker' id_employee, app_account_password_hash = digest('PizzaWorker' id_employee, 'sha256'); ALTER TABLE employee ALTER COLUMN app_account_password_hash SET NOT NULL; </pre>	Статистика 1 <table> <tr> <th>Name</th><th>Value</th></tr> <tr> <td>Updated Rows</td><td>7</td></tr> <tr> <td>Execute time</td><td>0,028s</td></tr> <tr> <td>Start time</td><td>Mon Dec 08 09:39:45 MSK 2025</td></tr> <tr> <td>Finish time</td><td>Mon Dec 08 09:39:45 MSK 2025</td></tr> <tr> <td>Query</td><td> UPDATE employee SET password_raw = 'PizzaWorker' id_employee, app_account_password_hash = digest('PizzaWorker' id_employee, 'sha256') </td></tr> </table>	Name	Value	Updated Rows	7	Execute time	0,028s	Start time	Mon Dec 08 09:39:45 MSK 2025	Finish time	Mon Dec 08 09:39:45 MSK 2025	Query	UPDATE employee SET password_raw = 'PizzaWorker' id_employee, app_account_password_hash = digest('PizzaWorker' id_employee, 'sha256')
Name	Value												
Updated Rows	7												
Execute time	0,028s												
Start time	Mon Dec 08 09:39:45 MSK 2025												
Finish time	Mon Dec 08 09:39:45 MSK 2025												
Query	UPDATE employee SET password_raw = 'PizzaWorker' id_employee, app_account_password_hash = digest('PizzaWorker' id_employee, 'sha256')												

Рисунок 17 – Заполнение паролей с хэшами

Рисунок 18 – Проверка паролей

2 Секционирование таблицы логов

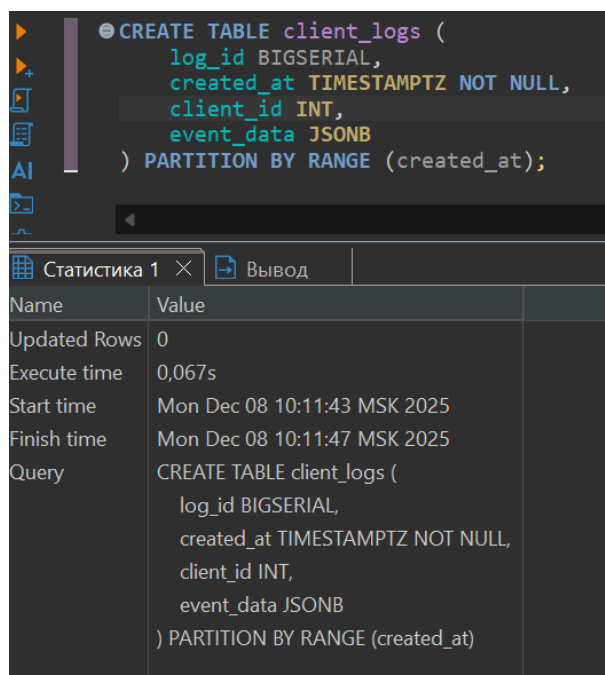


Рисунок 19 – Создание таблицы логов

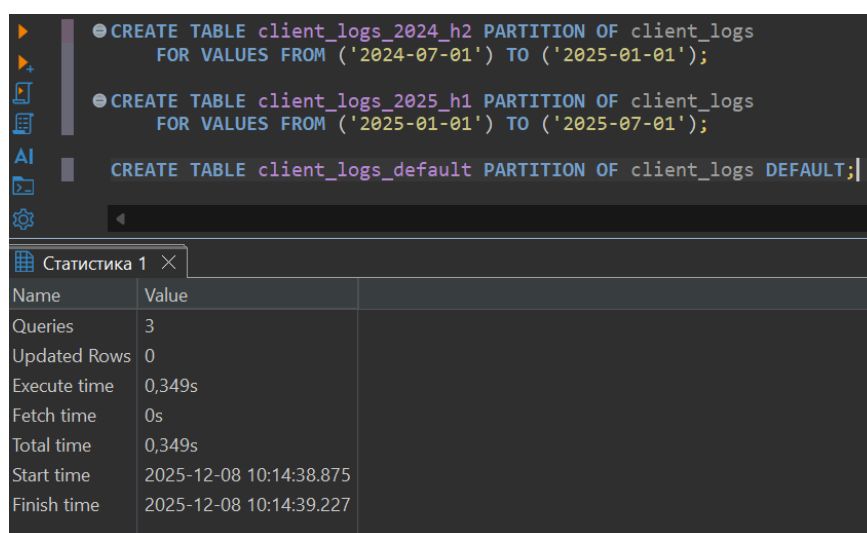


Рисунок 20 – Создание партиций

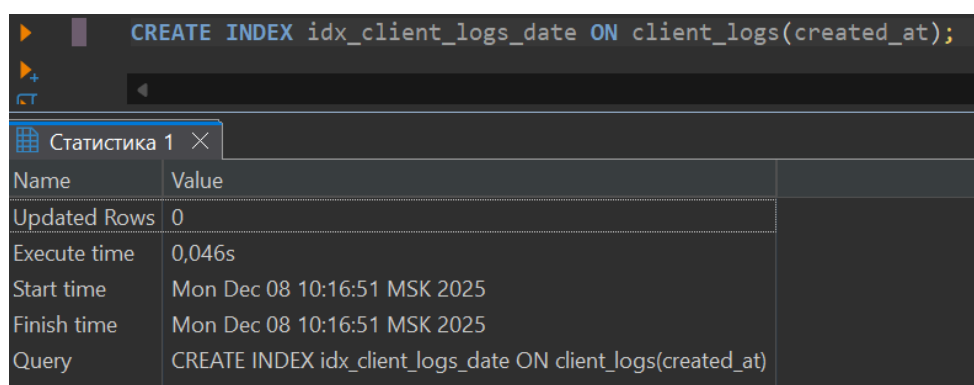


Рисунок 21 – Создание индекса

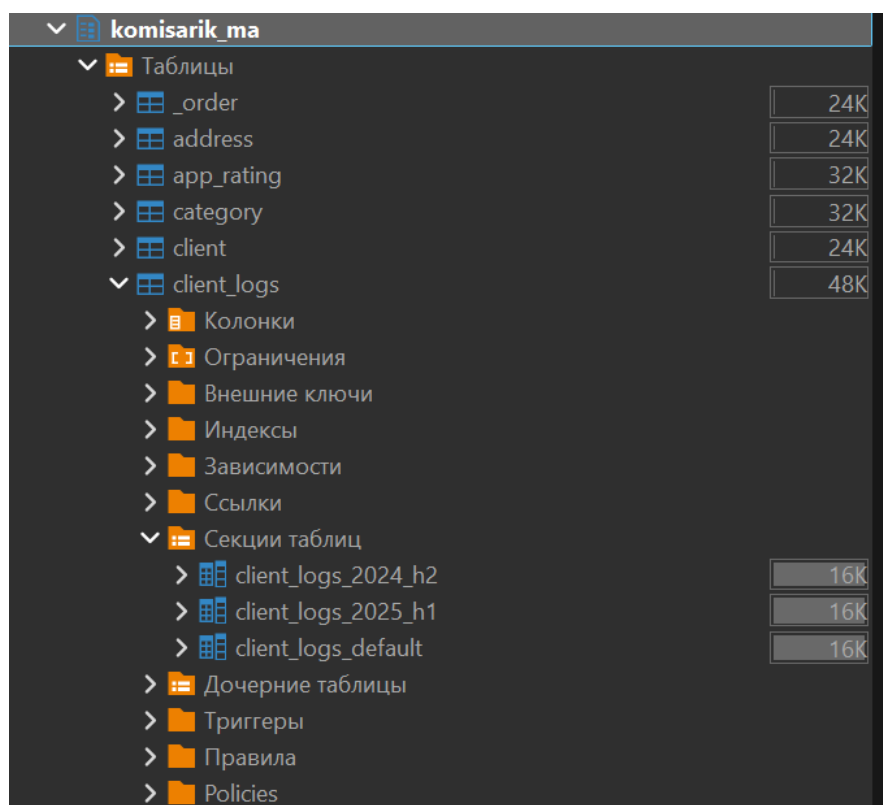


Рисунок 22 – Секции таблицы client_logs

3 Генерация данных

```
DO $$
DECLARE
  i INT;
  gen_date TIMESTAMPTZ;
  gen_client INT;
  event_type TEXT;
  json_payload JSONB;
BEGIN
  FOR i IN 1..20000 LOOP
    gen_date := '2024-07-01'::TIMESTAMPTZ + random() * ('2025-07-01'::TIMESTAMPTZ - '2024-07-01'::TIMESTAMPTZ);
    gen_client := floor(random() * 7 + 1)::INT;

    IF (i % 2 = 0) THEN
      event_type := 'login';
      json_payload := jsonb_build_object(
        'event', event_type,
        'ip', '192.168.0.' || floor(random() * 255)::TEXT,
        'success', (random() > 0.1)
      );
    ELSE
      event_type := 'sale';
      json_payload := jsonb_build_object(
        'event', event_type,
        'details', jsonb_build_object(
          'product_id', floor(random() * 11 + 1),
          'quantity', floor(random() * 5 + 1),
          'total_sum', (random() * 100)::NUMERIC(10,2)
        )
      );
    END IF;

    INSERT INTO client_logs (created_at, client_id, event_data)
    VALUES (gen_date, gen_client, json_payload);
  END LOOP;
END $$;
```

Name	Value
Updated Rows	0
Execute time	0,484s
Start time	Mon Dec 08 10:52:28 MSK 2025
Finish time	Mon Dec 08 10:52:28 MSK 2025
Query	DO \$\$ DECLARE i INT; gen_date TIMESTAMPTZ; gen_client INT;

Рисунок 23 – Скрипт генерации данных

```
SELECT '2024_h2' as partition, count(*) FROM client_logs_2024_h2
UNION ALL
SELECT '2025_h1', count(*) FROM client_logs_2025_h1
UNION ALL
SELECT 'default', count(*) FROM client_logs_default;
```

Таблица	А:Z partition	123 count
1	2024_h2	10 153
2	2025_h1	9 847
3	default	0

Рисунок 24 – Проверка генерации данных

4 Анализ и поиск по JSONB

```
SELECT log_id, event_data
FROM client_logs
WHERE event_data ->> 'event' = 'sale'
AND (event_data ->> 'details' ->> 'total_sum')::NUMERIC > 80
LIMIT 5;
```

client_logs 1 X

CREATE INDEX idx_client_logs_date ON client_logs (

	123 log_id	{ } event_data
1	17	{"event": "sale", "details": {"quantity": 2, "total_sum": 89.19, "product_id": 1}}
2	51	{"event": "sale", "details": {"quantity": 5, "total_sum": 99.69, "product_id": 5}}
3	57	{"event": "sale", "details": {"quantity": 2, "total_sum": 89.01, "product_id": 7}}
4	71	{"event": "sale", "details": {"quantity": 3, "total_sum": 85.36, "product_id": 5}}
5	79	{"event": "sale", "details": {"quantity": 2, "total_sum": 85.62, "product_id": 11}}

Рисунок 25 – Фильтрация по значению

```
SELECT log_id, created_at, event_data
FROM client_logs
WHERE event_data @> '{"details": {"quantity": 3}}'
LIMIT 5;
```

client_logs 1 X

SELECT log_id, created_at, event_data FROM client_logs

	123 log_id	created_at	{ } event_data
1	15	2024-08-31 20:41:01.822 +0300	{"event": "sale", "details": {"quantity": 3, "total_sum": 61.31, "product_id": 2}}
2	25	2024-07-04 09:30:01.526 +0300	{"event": "sale", "details": {"quantity": 3, "total_sum": 19.97, "product_id": 8}}
3	53	2024-10-18 02:08:53.024 +0300	{"event": "sale", "details": {"quantity": 3, "total_sum": 31.51, "product_id": 11}}
4	71	2024-08-28 07:37:43.715 +0300	{"event": "sale", "details": {"quantity": 3, "total_sum": 85.36, "product_id": 5}}
5	95	2024-07-19 16:27:02.778 +0300	{"event": "sale", "details": {"quantity": 3, "total_sum": 97.88, "product_id": 1}}

Рисунок 26 – Поиск по вхождению

```
SELECT
  AVG((event_data ->> 'details' ->> 'total_sum')::NUMERIC)::NUMERIC(10,2) AS avg_check
FROM client_logs
WHERE event_data ->> 'event' = 'sale';
```

результат 1 X

SELECT AVG(event_data ->> 'details' ->> 'total_sum')

	123 avg_check
1	50,43

Рисунок 27 – Подсчет среднего значения

5 Модификация данных JSONB

The screenshot shows a SQL query in a database client interface. The query is:

```
UPDATE client_logs
SET event_data = event_data || '{"source": "POS-terminal"}'::jsonb;

SELECT event_data FROM client_logs LIMIT 3;
```

Below the query, the results are displayed in a table with two columns: `log_id` and `event_data`. The first three rows are shown:

log_id	event_data
1	{"ip": "192.168.0.17", "event": "login", "source": "POS-terminal", "success": true}
2	{"event": "sale", "source": "POS-terminal", "details": {"quantity": 2, "total_sum": 44.13, "product_id": 8}}
3	{"ip": "192.168.0.228", "event": "login", "source": "POS-terminal", "success": true}

Рисунок 28 – Массовое изменение

The screenshot shows a SQL query in a database client interface. The query is:

```
select
  log_id,
  event_data
FROM client_logs
WHERE event_data ->> 'event' = 'login'
AND event_data ->> 'success' = 'false'
limit 1;
```

Below the query, the results are displayed in a table with two columns: `log_id` and `event_data`. The first row is shown:

log_id	event_data
16	{"ip": "192.168.0.196", "event": "login", "source": "POS-terminal", "success": false}

Рисунок 29 – Поиск записи для изменения статуса

The screenshot shows a SQL query in a database client interface. The query is:

```
UPDATE client_logs
SET event_data = jsonb_set(event_data, '{success}', 'true')
WHERE log_id = 16;

select
  log_id,
  event_data
FROM client_logs
WHERE log_id = 16;
```

Below the query, the results are displayed in a table with two columns: `log_id` and `event_data`. The first row is shown:

log_id	event_data
16	{"ip": "192.168.0.196", "event": "login", "source": "POS-terminal", "success": true}

Рисунок 30 – Точечное изменение